Autores: Tomás Ferreira (52737) / Tiago Pedroso (52915) / Tiago Inês (54193)

O módulo *Keyboard Reader* é constituído por três blocos principais: *i*) o descodificador de teclado (*Key Decode*); *ii*) o bloco de armazenamento (designado por *Ring Buffer*); e *iii*) o bloco de entrega ao consumidor (designado por *Key Transmitter*), de acordo com o diagrama representado na Figura 1. Neste caso o módulo *Control*, implementado em software, é a entidade consumidora.

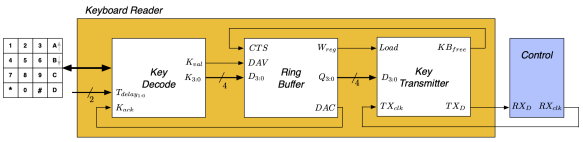
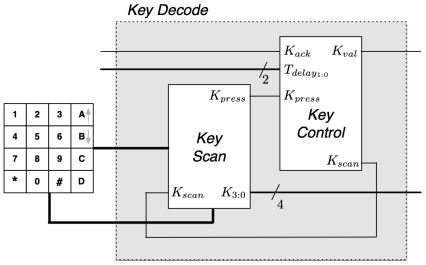
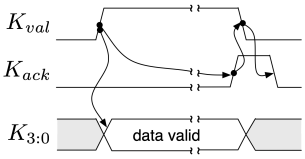


Figura 1: Diagrama de blocos do módulo *Keyboard Reader*



(a) Diagrama de blocos



(b) Diagrama temporal

Figura 2: Bloco *Key Decode*

1 Key Decode

O bloco *Key Decode* implementa um descodificador de um teclado matricial 4x4 por hardware, sendo constituído por três sub-blocos: *i*) um teclado matricial de 4x4; *ii*) o bloco Key Scan, responsável pelo varrimento do teclado; e *iii*) o bloco Key Control, que realiza o controlo do varrimento e o controlo de fluxo, conforme o diagrama de blocos representado na Figura 2a.

O controlo de fluxo de saída do bloco *Key Decode* (para o módulo *Key Buffer*), define que o sinal K_{val} é ativado quando é detetada a pressão de uma tecla, sendo também disponibilizado o código dessa tecla no barramento $K_{3:0}$. Apenas é iniciado um novo ciclo de varrimento ao teclado quando o sinal K_{ack} for ativado e a tecla premida for libertada. O diagrama temporal do controlo de fluxo está representado na Figura 2b.

O bloco *Key Scan* foi implementado de acordo com o diagrama de blocos representado na Figura 3. Escolhemos a opção 1 do Diagrama de blocos do bloco Key Scan porque é a solução que nos pareceu mais adequada para implementar o varrimento do teclado nesta fase. Esta opção permite um varrimento sequencial direto das linhas e das colunas do teclado matricial, o que torna o funcionamento mais previsível e mais fácil de controlar, o que facilita a deteção correta de teclas e o cumprimento do protocolo de controlo (Kval/Kack).

Além disso, apresenta menor complexidade de hardware, usando menos componentes lógicos, o que reduz a probabilidade de ocorrerem erros e simplifica a implementação.

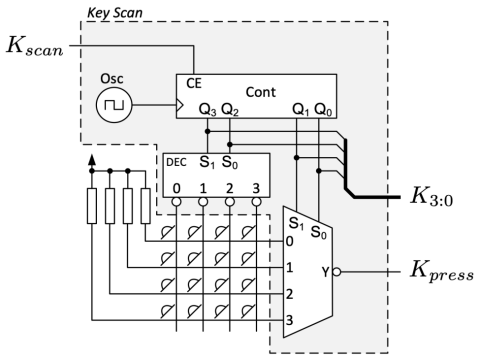


Figura 3: Diagrama de blocos do módulo *Key Scan*

O bloco *Key Control* foi implementado pela máquina de estados representada em *ASM-chart* na Figura 4.

Descrição da Máquina de estados do Key Control: No estado 00, o Key Scan é ativado. Se o Key Press estiver a 1, vai para o estado 01 onde é ativado o Key Val, enquanto que se o Key Press estiver a 0, vai manter-se no estado 00. No estado 01, o Key Val é ativado. Se o Key Ack esiver a 1, vai para o estado 10, onde o dado é lido, enquanto que se o Key Ack estiver a 0, vai manter-se no estado 01.

No estado 10, o dado vai ser lido enquanto a tecla estiver a ser pressionada e o key act estiver ativo. Se o Key Press estiver a 1, vai manter-se no estado 01 enquanto que se o Key Press estiver a 0, o estado seguinte irá depender do Key Ack. Se o Key Ack estiver a 1, vai manter-se no estado 10, enquanto que se o Key Ack estiver a 0 vai para o estado 00.

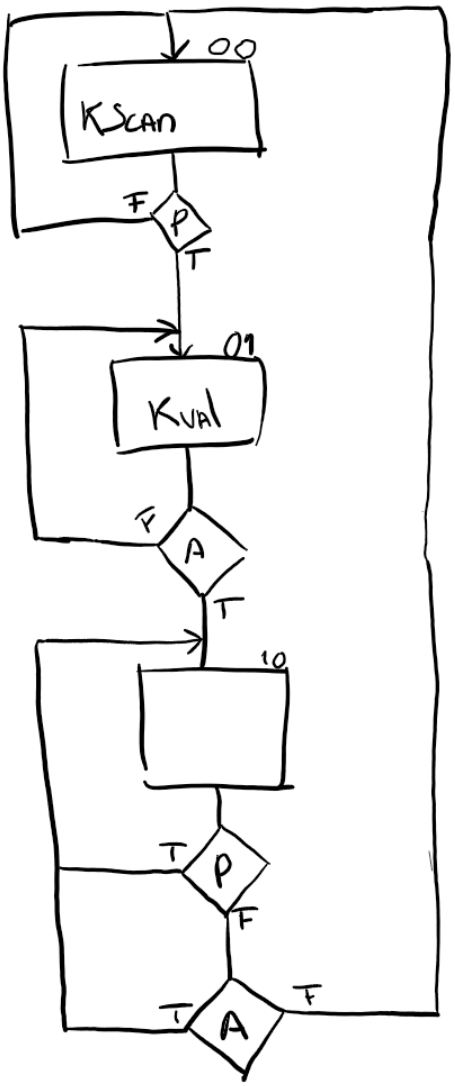


Figura 4: Máquina de estados do *Key Control*

A descrição hardware do bloco *Key Decode* em *VHDL* encontra-se no Anexo A.1.

2 Conclusões

O desenvolvimento do módulo Keyboard Reader permitiu compreender e implementar a leitura de um teclado matricial 4x4 através de uma abordagem estruturada e eficiente. O foco no bloco Key Decode revelou-se suficiente para garantir a deteção correta das teclas premidas, evidenciando a importância de um varrimento sequencial bem controlado.

O mecanismo de controlo de fluxo, baseado nos sinais Kval e Kack, assegura uma comunicação fiável, evitando leituras incorretas ou repetidas.

De forma geral, a solução apresenta baixa complexidade e boa previsibilidade, cumprindo os objetivos propostos. Este trabalho permitiu consolidar conceitos fundamentais como o varrimento de teclados matriciais e o controlo síncrono, constituindo uma base importante para futuros desenvolvimentos em sistemas digitais.

A VHDL

A.1 Key Decode

```
library ieee;
use ieee.std_logic_1164.all;

entity Key_Decode is
    port (
        Kack, RESET, CLK : in  std_logic;
        Tdelay           : in  std_logic_vector(1 downto 0);
        Kval             : out std_logic;
        K                : out std_logic_vector(3 downto 0);
        Keys_Vertical    : out std_logic_vector(3 downto 0);
        Keys_Horizontal  : in  std_logic_vector(3 downto 0)
    );
end entity Key_Decode;

architecture logicFunction of Key_Decode is

    component Key_Control is
        port (
            Kack, Kpress, RESET, CLK : in  std_logic;
            Tdelay                   : in  std_logic_vector(1 downto 0);
            Kval, Kscan              : out std_logic
        );
    end component Key_Control;

    component Key_Scan is
        port (
            Kscan, CLK, RESET : in  std_logic;
            Keys_Vertical     : out std_logic_vector(3 downto 0);
            Keys_Horizontal   : in  std_logic_vector(3 downto 0);
            K                 : out std_logic_vector(3 downto 0);
            Kpress            : out std_logic
        );
    end component Key_Scan;

    component CLKDIV is
        generic (
            div     :     natural := 50000000
        );
        port (
            clk_in  : in  std_logic; -- Entrada do clock div
            clk_out : out std_logic  -- Saída do clock div
        );
    end component CLKDIV;


    signal Kpress, Kscan, not_clk_divider, CLK_Divider : std_logic;

begin

    not_clk_divider <= not CLK_Divider;

    scan: component Key_Scan
    port map (
        RESET           => RESET,
        CLK             => CLK_Divider,
        Kpress          => Kpress,
        K               => K,
        Kscan           => Kscan,
        Keys_Vertical   => Keys_Vertical,
        Keys_Horizontal => Keys_Horizontal
```

```
    );

    control: component Key_Control
    port map (
        RESET           => RESET,
        CLK             => CLK_Divider,
        Kpress          => Kpress,
        Kack            => Kack,
        Tdelay          => Tdelay,
        Kval            => Kval,
        Kscan           => Kscan
    );

    Clk_div: component CLKDIV
    generic map (
        50
    )
    port map (
        clk_in          => CLK,
        clk_out         => CLK_Divider
    );


end architecture logicFunction;
```

A.2 Key Scan

```
library IEEE;
use IEEE.std_logic_1164.all;

entity Key_Scan is

    port (
        Kscan, CLK, RESET : in  std_logic;
        Keys_Vertical     : out std_logic_vector(3 downto 0);
        Keys_Horizontal   : in  std_logic_vector(3 downto 0);
        K                 : out std_logic_vector(3 downto 0);
        Kpress            : out std_logic
    );

end entity Key_Scan;

architecture scan of Key_Scan is

    component Counter is
        port (
            CE    : in  std_logic;
            CLK   : in  std_logic;
            Q     : out std_logic_vector(3 downto 0);
            RESET : in  std_logic
        );
    end component Counter;

    component Decoder is
        port (
            S : in  std_logic_vector(1 downto 0);
            C : out std_logic_vector(3 downto 0)
        );
    end component Decoder;

    component MUX4X1 is
        port (
            A  : in  std_logic_vector(3 downto 0);
            OP : in  std_logic_vector(1 downto 0);
            F  : out std_logic
        );
    end component MUX4X1;

    signal counter_out, not_c : std_logic_vector(3 downto 0);

    signal not_k_press        : std_logic;
begin

    Kcounter: component Counter
    port map (
        CLK   => CLK,
        RESET => RESET,
        CE    => Kscan,
        Q     => counter_out
    );

    mux: component MUX4X1
    port map (
        A     => Keys_Horizontal,
        OP    => counter_out(1 downto 0),
        F     => not_k_press
    );

    SKdecoder: component Decoder
```

```
    port map (
        S     => counter_out(3 downto 2),
        C     => not_c
    );

    Kpress        <= not not_k_press;
    Keys_Vertical <= not not_c;
    K             <= counter_out;

end architecture scan;
```

A.3 Key Decoder FSM

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity KeyDecoderFSM is
    port (
        reset        : in  std_logic;
        clk          : in  std_logic;
        Kpress, Kack : in  std_logic;
        Tdelay       : in  std_logic_vector(1 downto 0); -- 00=500ms 01=1000ms 10=1500ms 11=2000ms
        Kval, Kscan  : out std_logic
    );
end entity KeyDecoderFSM;

architecture behavioral of KeyDecoderFSM is

    component KeyDelay is
        port (
            CLK     : in  std_logic;
            Tdelay  : in  std_logic_vector(1 downto 0);
            CLK_Out : out std_logic
        );
    end component KeyDelay;

    type STATE_TYPE is (STANDING_BY, READING_DATA, DATA_ACCEPTED);
    signal CurrentState, NextState : STATE_TYPE;
    signal clk_out                 : std_logic;
    signal clk_out_prev            : std_logic := '0';
    signal clk_out_rise            : std_logic;

begin

    -- Instantiate delay clock generator
    clkd: component KeyDelay
    port map (
        CLK     => clk,
        Tdelay  => Tdelay,
        CLK_Out => clk_out
    );

    DetectRise: process (clk, reset) is
    begin
        if reset = '1' then
            clk_out_prev      <= '0';
        elsif rising_edge(clk) then
            clk_out_prev      <= clk_out;
        end if;
    end process DetectRise;

    clk_out_rise <= '1' when (clk_out = '1' and clk_out_prev = '0') else '0';

        CurrentState <= STANDING_BY when reset = '1' else NextState;


        GenerateNextState: process (CurrentState, Kpress, Kack, clk_out_rise) is
    begin
        case CurrentState is

            when STANDING_BY =>
                if (Kpress = '1') then
                    NextState <= READING_DATA;
                else
```

```
                NextState <= STANDING_BY;
            end if;

        when READING_DATA =>
            if (Kack = '1') then
                NextState <= DATA_ACCEPTED;
            else
                NextState <= READING_DATA;
            end if;

        when DATA_ACCEPTED =>
            if (clk_out_rise = '1') or (Kack = '0' and Kpress = '0') then
                NextState <= STANDING_BY;
            else
                NextState <= DATA_ACCEPTED;
            end if;

    end case;
end process GenerateNextState;

Kscan <= '1' when (CurrentState = STANDING_BY) else '0';
Kval <= '1' when (CurrentState = READING_DATA) else '0';

end architecture behavioral;
```

A.4 Key Control

```
library ieee;
use ieee.std_logic_1164.all;

entity Key_Control is
    port (
        Kack, Kpress, RESET, CLK : in  std_logic;
        Tdelay                   : in  std_logic_vector(1 downto 0);
        Kval, Kscan              : out std_logic
    );
end entity Key_Control;

architecture logicFunction of Key_Control is

    component KeyDecoderFSM is
        port (
            reset        : in  std_logic;
            clk          : in  std_logic;
            Kpress, Kack : in  std_logic;
            Tdelay       : in  std_logic_vector(1 downto 0);
            Kval, Kscan  : out std_logic
        );
    end component KeyDecoderFSM;

    signal k_val : std_logic;

begin

    fsm: component KeyDecoderFSM
    port map (
        reset  => RESET,
        clk    => CLK,
        Kpress => Kpress,
        Kack   => Kack,
        Tdelay => Tdelay,
        Kval   => Kval,
        Kscan  => Kscan
    );
end architecture logicFunction;
```

A.5 Key Delay

```
library ieee;
use ieee.std_logic_1164.all;

entity KeyDelay is
    port (
        CLK     : in  std_logic;
        Tdelay  : in  std_logic_vector(1 downto 0);
        CLK_Out : out std_logic
    );
end entity KeyDelay;

architecture logicFunction of KeyDelay is

    component MUX4X1 is
        port (
            A  : in  std_logic_vector(3 downto 0);
            OP : in  std_logic_vector(1 downto 0);
            F  : out std_logic
        );
    end component MUX4X1;

    component CLKDIV is
        generic (
            div     :     natural := 50000000
        );
        port (
            clk_in  : in  std_logic; -- Entrada do clock div
            clk_out : out std_logic  -- Saída do clock div
        );
    end component CLKDIV;


    signal CLK_Divider_00, CLK_Divider_01, CLK_Divider_10, CLK_Divider_11 : std_logic;

begin

    Clk_div_00: component CLKDIV
    generic map (
        500000
    )
    port map (
        clk_in  => CLK,
        clk_out => CLK_Divider_00
    );

    Clk_div_01: component CLKDIV
    generic map (
        1000000
    )
    port map (
        clk_in  => CLK,
        clk_out => CLK_Divider_01
    );

    Clk_div_10: component CLKDIV
    generic map (
        1500000
    )
    port map (
        clk_in  => CLK,
        clk_out => CLK_Divider_10
    );
```

```
    Clk_div_11: component CLKDIV
    generic map (
        2000000
    )
    port map (
        clk_in  => CLK,
        clk_out => CLK_Divider_11
    );

    mux: component MUX4X1
    port map (
        A(0)    => CLK_Divider_00,
        A(1)    => CLK_Divider_01,
        A(2)    => CLK_Divider_10,
        A(3)    => CLK_Divider_11,
        OP      => Tdelay,
        F       => CLK_Out
    );

end architecture logicFunction;
```

B *Kotlin*

B.1 HAL

Algoritmo 1: HAL - Hardware Abstract Layer

```
import isel.leic.UsbPort

object HAL {
    private var bufferWrite = 0

    fun init() {
        UsbPort.write(bufferWrite)
    }

    fun isBit(mask: Int): Boolean {
        return (UsbPort.read() and mask) != 0
    }

    fun readBits(mask: Int): Int {
        return UsbPort.read() and mask
    }

    fun writeBits(mask: Int, value: Int) {
        bufferWrite = (bufferWrite and mask.inv()) or (value and mask)
        UsbPort.write(bufferWrite)
    }

    fun setBits(mask: Int) {
        bufferWrite = bufferWrite or mask
        UsbPort.write(bufferWrite)
    }

    fun clrBits(mask: Int) {
        bufferWrite = bufferWrite and mask.inv()
        UsbPort.write(bufferWrite)
    }
}
```

B.2 KBD

Algoritmo 2: KBD

```
object KBD {
    private const val KVAL_MASK = 0x80
    private const val KCODE_MASK = 0x0F

    private const val KACK_MASK = 0x80

    enum class Key(val char: Char) {
        KEY_1('1'),
        KEY_2('2'),
        KEY_3('3'),
        KEY_4('4'),
        KEY_5('5'),
        KEY_6('6'),
        KEY_7('7'),
        KEY_8('8'),
        KEY_9('9'),
        KEY_0('0'),
        KEY_A('A'),
        KEY_B('B'),
        KEY_C('C'),
        KEY_D('D'),
        KEY_HASH('#'),
        KEY_STAR('*'),
        KEY_NONE('\u0000')
    }


    fun init() {
```

```
        HAL.init()
    }

    fun getKey(): Char {
        return readKey().char
    }

    fun readKey(): Key {
        if (!HAL.isBit(KVAL_MASK)) return Key.KEY_NONE

        val code = HAL.readBits(KCODE_MASK)
        HAL.setBits(KACK_MASK)

        while (HAL.isBit(KVAL_MASK)) {
            Thread.yield()
        }

        HAL.clrBits(KACK_MASK)

        return convertToKey(code)
    }

    private fun convertToKey(id: Int): Key = when (id) {
        0b0000 -> Key.KEY_1
        0b0100 -> Key.KEY_2
        0b1000 -> Key.KEY_3
        0b1100 -> Key.KEY_A
        0b0001 -> Key.KEY_4
        0b0101 -> Key.KEY_5
        0b1001 -> Key.KEY_6
        0b1101 -> Key.KEY_B
        0b0010 -> Key.KEY_7
        0b0110 -> Key.KEY_8
        0b1010 -> Key.KEY_9
        0b1110 -> Key.KEY_C
        0b0111 -> Key.KEY_STAR
        0b1011 -> Key.KEY_HASH
        0b0011 -> Key.KEY_0
        0b1111 -> Key.KEY_D
        else -> Key.KEY_NONE
    }

    fun waitKey(timeout: Long): Char {
        val endTime = System.currentTimeMillis() + timeout

        while (System.currentTimeMillis() < endTime) {
            val key = getKey()
            if (key != Key.KEY_NONE.char) return key
            Thread.sleep(1)
        }
        return Key.KEY_NONE.char
    }
}
```